

INFORME TÉCNICO QUERIES Y PATCH EN SUPABASE

- Elaborado por:
- Nicolas Rojas
- Nikol Ortiz
- Santiago Dehoyos
- Danna Ortiz

Introducción

Este informe describe las consultas (**Queries**) y actualizaciones (**Patch**) realizadas sobre la tabla usuario utilizando la API REST de Supabase.

El código utiliza fetch para conectarse a la base de datos mediante las variables de entorno:

- VITE_SUPABASE_URL
- VITE_SUPABASE_API_KEY

Además, se configuran los encabezados necesarios para autenticar y enviar datos en formato JSON.

Configuración General

```
src > components > JS apis.js > getServicios
1  const SUPABASE_URL = import.meta.env.VITE_SUPABASE_URL;
2  const SUPABASE_KEY = import.meta.env.VITE_SUPABASE_API_KEY;
3
4  const headers = {
5    "Content-Type": "application/json",
6    "apikey": SUPABASE_KEY,
7    "Authorization": `Bearer ${SUPABASE_KEY}`,
8  };
9
10
```

Explicación

- SUPABASE_URL: almacena la URL del proyecto de Supabase.
- SUPABASE_KEY: contiene la clave de autenticación.
- headers: define el tipo de contenido y la autorización para acceder a la API.

QUERIES

Query 1 – Obtener todos los usuarios

```

11
12 // Query 1 Obtener todos los registros de la tabla usuario
13
14 export async function getServicios() {
15
16     const res = await fetch(
17         `${SUPABASE_URL}/rest/v1/usuario?select=*`,
18         {
19             method: "GET",
20             headers,
21         }
22     );
23
24     if (!res.ok) {
25         throw new Error("Error al obtener usuarios");
26     }
27
28     return await res.json();
29 }
30

```

Explicación

Esta consulta obtiene todos los registros de la tabla usuario.

Método HTTP: GET

Resultado esperado: Devuelve un arreglo con todos los usuarios registrados.

Query 2 – Obtener un usuario por ID

```

33 // Query 2 Obtener un usuario por ID
34
35 export async function getUsuarioPorId(id) {
36
37     const res = await fetch(
38         `${SUPABASE_URL}/rest/v1/usuario?id=eq.${id}&select=*`,
39         {
40             method: "GET",
41             headers,
42         }
43     );
44
45     if (!res.ok) {
46         throw new Error("Error al obtener usuario");
47     }
48
49     return await res.json();
50 }

```

Explicación:

Busca un usuario específico utilizando su ID.

Parámetro:

id: identificador único del usuario.

Resultado esperado: Devuelve la información del usuario correspondiente.

Query 3 – Buscar usuarios por nombre

```
54 // Query 3  Buscar usuarios por nombre
55
56 export async function buscarUsuario(nombre) {
57
58   const res = await fetch(
59     `${SUPABASE_URL}/rest/v1/usuario?nombre=ilike.%${nombre}%&select=*`,
60     {
61       method: "GET",
62       headers,
63     }
64   );
65
66   if (!res.ok) {
67     throw new Error("Error al buscar usuario");
68   }
69
70   return await res.json();
71 }
72
```

Explicación

Permite buscar usuarios cuyo nombre coincida parcialmente con el texto ingresado.

Operador utilizado

ilike

- Realiza búsquedas sin importar mayúsculas o minúsculas.

Resultado esperado: Devuelve los usuarios encontrados.

Query 4 – Obtener usuario por correo electrónico:

```
75 // Query 4  Obtener usuario por correo electronico
76
77 export async function getUsuarioCorreo(correo) {
78
79   const res = await fetch(
80     `${SUPABASE_URL}/rest/v1/usuario?correo=eq.${correo}&select=*`,
81     {
82       method: "GET",
83       headers,
84     }
85   );
86
87   if (!res.ok) {
88     throw new Error("Error al obtener correo");
89   }
90
91   return await res.json();
92 }
93
```

Explicación

Busca un usuario utilizando su correo electrónico.

Resultado esperado: Devuelve el usuario asociado al correo ingresado.

Query 5 – Obtener usuarios activos

```
// Query 5  Obtener usuarios activos

export async function getUsuariosActivos() {

  const res = await fetch(
    `${SUPABASE_URL}/rest/v1/usuario?activo=eq.true&select=*`,
    {
      method: "GET",
      headers,
    }
  );

  if (!res.ok) {
    throw new Error("Error al obtener usuarios activos");
  }

  return await res.json();
}
```

Explicación

Obtiene únicamente los usuarios cuyo estado sea activo (true).

Resultado esperado:

Devuelve una lista de usuarios activos.

PATCH:

Los métodos PATCH permiten actualizar información específica de un usuario.

Patch 1 – Actualizar todos los datos de un usuario

```
// Patch 1 Actualizar todos los datos de un usuario

export async function actualizarServicio(id, cambios) {

  const res = await fetch(
    `${SUPABASE_URL}/rest/v1/usuario?id=eq.${id}`,
    {
      method: "PATCH",
      headers: {
        ...headers,
        Prefer: "return=representation",
      },
      body: JSON.stringify(cambios),
    }
  );

  if (!res.ok) {
    throw new Error("Error al actualizar usuario");
  }

  const data = await res.json();

  return data[0];
}
```

Explicación

Actualiza varios campos de un usuario al mismo tiempo utilizando un objeto cambios.

Resultado esperado:

Devuelve el usuario actualizado.

Patch 2 – Actualizar el nombre de un usuario

```
// Patch 2 Actualizar el nombre de un usuario

export async function actualizarNombre(id, nombre) {

  const res = await fetch(
    `${SUPABASE_URL}/rest/v1/usuario?id=eq.${id}`,
    {
      method: "PATCH",
      headers: {
        ...headers,
        Prefer: "return=representation",
      },
      body: JSON.stringify({ nombre }),
    }
  );

  if (!res.ok) {
    throw new Error("Error al actualizar nombre");
  }

  return await res.json();
}
```

Función:

```
146 export async function actualizarNombre(id, nombre) {
```

Explicación:

Modifica únicamente el nombre del usuario.

Patch 3 – Actualizar el correo de un usuario

```
// Patch 3 Actualizar el correo de un usuario

export async function actualizarCorreo(id, correo) {

  const res = await fetch(
    `${SUPABASE_URL}/rest/v1/usuario?id=eq.${id}`,
    {
      method: "PATCH",
      headers: {
        ...headers,
        Prefer: "return=representation",
      },
      body: JSON.stringify({ correo }),
    }
  );

  if (!res.ok) {
    throw new Error("Error al actualizar correo");
  }

  return await res.json();
}
```

Función:

```
export async function actualizarCorreo(id, correo) {
```

Explicación

Actualiza el correo electrónico de un usuario.

Patch 4 – Cambiar el estado de un usuario

```
// Patch 4 Cambiar el estado activo de un usuario

export async function cambiarEstado(id, activo) {

  const res = await fetch(
    `${SUPABASE_URL}/rest/v1/usuario?id=eq.${id}`,
    {
      method: "PATCH",
      headers: {
        ...headers,
        Prefer: "return=representation",
      },
      body: JSON.stringify({ activo }),
    }
  );

  if (!res.ok) {
    throw new Error("Error al cambiar estado");
  }

  return await res.json();
}
```

Función:

```
export async function cambiarEstado(id, activo) {
```

Explicación:

Permite activar o desactivar un usuario.

Patch 5 – Actualizar la contraseña de un usuario

```
// Patch 5  Actualizar la contraseña de un usuario

export async function actualizarPassword(id, password) {

  const res = await fetch(
    `${SUPABASE_URL}/rest/v1/usuario?id=eq.${id}`,
    {
      method: "PATCH",
      headers: {
        ...headers,
        Prefer: "return=representation",
      },
      body: JSON.stringify({ password }),
    }
  );

  if (!res.ok) {
    throw new Error("Error al actualizar contraseña");
  }

  return await res.json();
}
```

Función

```
export async function actualizarPassword(id, password) {
```

Explicación:

Actualiza la contraseña de un usuario.

Manejo de errores:

Todas las funciones incluyen validación mediante:


```
if (!res.ok) {  
  throw new Error("Error al actualizar contraseña");  
}  
  
return await res.json();  
}
```

Explicación:

- Verifica si la petición fue exitosa.
- Si ocurre un error, se genera una excepción con un mensaje personalizado.

Conclusión:

Las consultas y actualizaciones implementadas permiten administrar la información de los usuarios en la base de datos de manera eficiente mediante la API REST de Supabase.

Las Queries se utilizan para obtener información, mientras que los métodos PATCH permiten modificar datos específicos sin reemplazar todo el registro. Esto facilita la gestión de usuarios dentro de la aplicación de forma rápida y organizada.